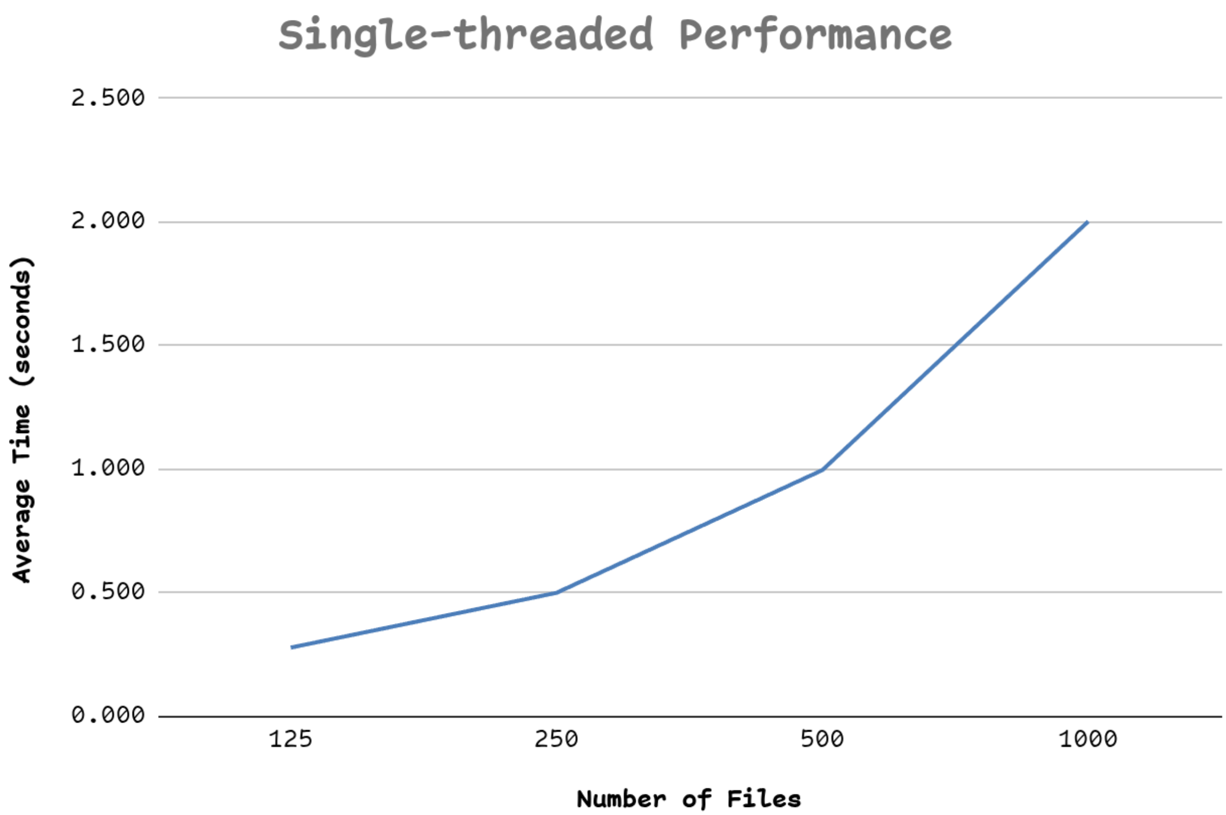
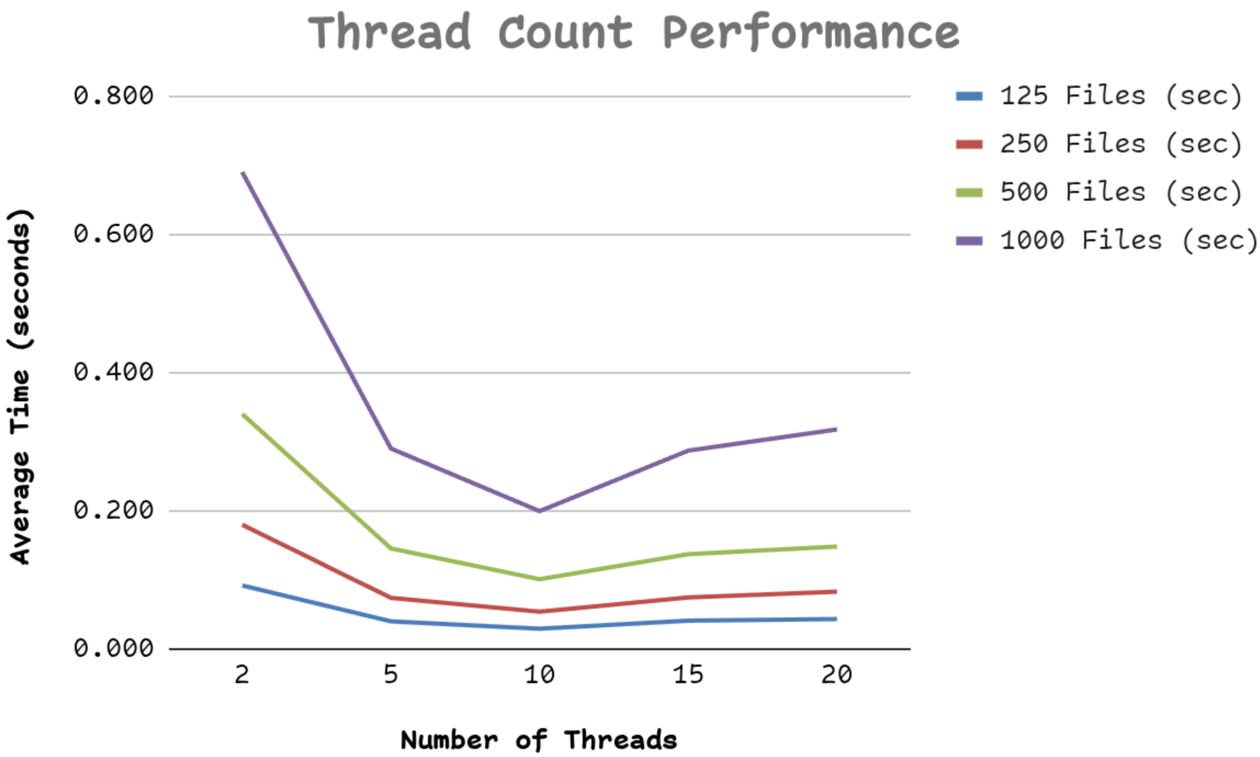
Full source code: https://github.com/lukedaoo/zod-threadscan
Experiment 1:



How does the search time increase with the number of files. Does the data make sense?
Answer:
The time grows in a straight line with the number of files. Look at the numbers: 125 files took ~0.28s, 250 files took ~0.50s, 500 files took ~0.99s, 1000 files took ~2.00s. Each time I double the files, the time doubles too.

This makes perfect sense. With one thread, the program reads files one after another. If reading one file takes a tiny bit of time, then reading twice as many files takes twice as much time. There is no shortcut. This is linear growth (O(n)).

Experiment 2:



Does increasing the thread count always improve performance? If your data shows otherwise (i.e., as the number of threads keeps increasing, at some point performance decreases), then what might be a possible reason?

Answer:

No. The data clearly shows there is a sweet spot, and after that, more threads actually make things ***slower***. For 1000 files: 2 threads -> 0.69s, 5 threads -> 0.29s, 10 threads -> 0.20s (fastest), 15 threads -> 0.29s, 20 threads -> 0.32s.

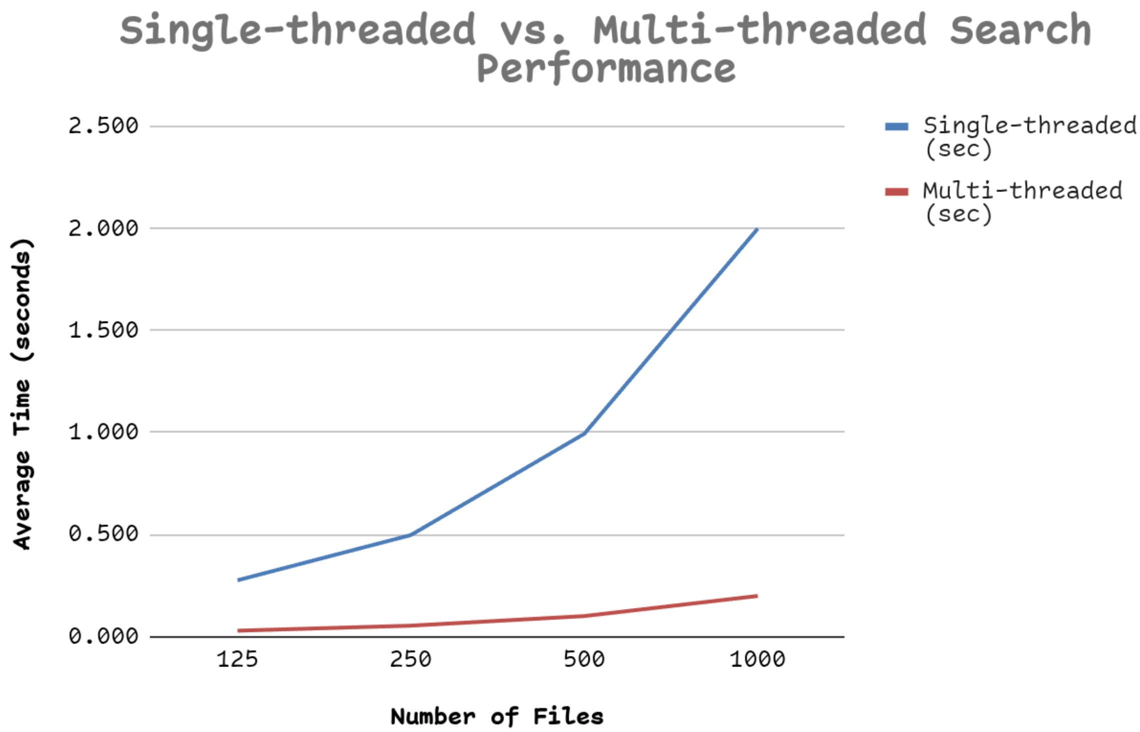
The pattern: speed improves up to 10 threads, then gets worse. Three likely reasons:

1. Too many threads, not enough CPU cores. My computer has a fixed number of cores (probably 8-12). If I create 20 threads but only have 10 cores, the operating system has to keep pausing one thread to let another run. This pausing and resuming (context switching) wastes time.

2. Overhead of creating and managing threads. Each new thread costs something to set up, coordinate, and clean up. With 20 threads splitting 125 files, each thread only handles ~6 files the setup cost starts to outweigh the actual work.

3. Threads fighting over shared resources. Threads share the disk, the cache, and any locks More threads means more waiting in line for these shared resources.

Experiment 3:



Does the concurrency brough about by multi-threading improves search time when compared with single-threaded performance? Why multiple threads are faster than a single one?
Answers:

Yes, by a huge margin, about 10x faster across every file size. The single-threaded program took 2.00s on 1000 files; the 10-thread version did it in 0.20s.

Why multiple threads win: modern CPUs have multiple cores, and each core can do real work at the same time. A single thread can only use one core, the other 9 just sit there doing nothing. With 10 threads, all 10 cores work in parallel. The reason the speedup is so clean (close to a perfect 10x) is that searching for a word in a file is parallel, each file can be scanned completely independently of the others. There's no need for threads to talk to each other or wait for results, so almost no time is wasted on coordination.